Sample Implementation

A complete, runnable reference host showing every feature of the base module — copy patterns from it directly into your project.

What's in the sample

Path	What it demonstrates
`dotnet/samples/SampleApi/Program.cs`	The full 3-line integration + production-hardening switches
`dotnet/samples/SampleApi/appsettings.json`	Complete `AiSuite` config template (providers, FAQ, 15-section system prompt, languages, rate limit, settings panel)
`dotnet/samples/SampleApi/wwwroot/demo.html`	Standalone frontend: popup chat widget, language switcher, streaming, and the password-gated settings panel (multi-key LLM, voice, knowledge uploads)
`samples/sample-requests.http`	Copy-paste requests for every endpoint (VS Code / Rider REST client)
`dotnet/tests/Pixous.AI.Tests.Unit`	76 unit tests (providers, fallback, store, parsers, prompt)
`dotnet/tests/Pixous.AI.Tests.Integration`	24 integration tests via `WebApplicationFactory` — run with no real keys

The backend (Program.cs — the whole integration)

```
using Pixous.AI.AspNetCore.Extensions;
using Pixous.AI.AspNetCore.Hosting;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAiSuite(builder.Configuration);   // 1. DI + options

var app = builder.Build();

if (app.Configuration.GetValue&lt;bool&gt;("AiSuite:RateLimit:Enabled"))
    app.UseRateLimiter();                             // optional hardening

app.UseStaticFiles();                                 // serves the demo page
app.MapAiSuiteEndpoints();                            // 2. /api/ai/* routes

app.Run();

public partial class Program { }                      // for WebApplicationFactory
```

That is the entire integration. Everything else — providers, resilience, runtime selection, settings panel, voice, knowledge base — arrives through `AddAiSuite` + configuration.

The configuration (appsettings.json)

The `AiSuite` section is the single control surface:

```
{
  "AiSuite": {
    "ChatProvider": "KnowledgeBase",          // Echo | KnowledgeBase | OpenAiCompatible
    "TranslationProvider": "Echo",            // Echo | AzureTranslator | LibreTranslate
    "OpenAiCompatible": {
      "Endpoint": "https://api.openai.com/v1",
      "Model": "gpt-4o-mini",
      "ApiKeyConfigKey": "AiSuite:ApiKeys:Chat"   // pointer — never the secret
    },
    "KnowledgeBase": { "Entries": [ /* FAQ */ ] },
    "ChatDefaults": { "SystemPrompt": "…15-section agent prompt…" },
    "Languages": { "Default": "en", "Supported": ["en","ta","hi","es","fr"] },
    "Settings": { "Enabled": true, "Password": "pixous" }
  }
```

```
}
```

Run it

```
cd dotnet
dotnet build Pixous.AI.slnx
dotnet test  Pixous.AI.slnx                # 76 + 24, no keys required
dotnet run --project samples/SampleApi --urls http://localhost:5199
```

Then open `http://localhost:5199/demo.html`:

1. Click the launcher → chat widget (type a message, try **Listen**, watch SSE streaming).
2. Click → password `pixous` → settings panel:
   - **Chat** tab — switch providers live (KnowledgeBase answers instantly with citations).
   - **LLM APIs** tab — add any vendor's key (masked forever after), reorder for fallback, test a key.
   - **Voice** tab — pick STT/TTS providers (Echo = keyless demo).
   - **Knowledge** tab — upload a `.md` / `.pdf` / `.docx` / `.txt` file and ask about it.
   - **Security** tab — change the settings password.
3. Fire the requests in `samples/sample-requests.http` to see raw envelopes.

Talking to it (envelope shape)

```
curl -X POST http://localhost:5199/api/ai/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"how much is the enterprise plan?"}'
# → { "success": true, "data": { "reply": "…", "provider": "KnowledgeBase" },
#     "error": null, "traceId": "…" }
```

Streaming (SSE) is the same body on `POST /api/ai/chat/stream`: `data: <delta>` frames until `data: [DONE]`.

Test patterns worth copying

- **Unit**: fake `IChatProvider` implementations; MultiLLM fallback driven with stub HTTP handlers (429 → cooldown, 401 → invalid); settings store against a temp directory; document parsers against tiny fixture files.
- **Integration**: `WebApplicationFactory{Program}` with `AiSuite:Settings:Password` overridden — login → `X-Settings-Token` → snapshot / key CRUD / voice / KB upload, all asserting the masked-key contract (raw keys never in responses).

Porting the sample to your project

1. Copy the `AiSuite` section into your `appsettings.json`.
2. Add the two integration lines to your `Program.cs`.
3. Reuse `demo.html` as-is for a quick start, or wire `@pixous/ai-suite` components into your Angular app (see `angular-integration.md`).
4. Point `ApiKeyConfigKey` values at your User Secrets / env / Key Vault.
5. Flip the hardening switches (auth, rate limit, CORS) per `security.md`.